

Single-Shot Autocorrelator Design and Analysis for Ultrafast Laser Diagnostics

Josh Kazi

University of Waterloo

20838592

December 2, 2024

*Submitted in partial fulfillment of the requirements for Physics
437A*

Contents

1	Introduction and History	3
1.1	Ultrashort Laser Pulses and Their Measurement Challenges	3
1.2	Development of Autocorrelation Techniques	3
1.3	The Single-Shot Autocorrelator	4
2	Experimental Techniques	5
2.1	Single-Shot Autocorrelator Setup	5
2.2	Beam Alignment	6
2.3	Delay Stage	7
2.3.1	Derivation of the Temporal Calibration Factor	7
2.3.2	Symmetry of the Autocorrelation and Stray Light	8
2.3.3	Energy/Frequency in SHG (for completeness)	8
2.4	Nonlinear Crystal and Phase Matching	9
2.4.1	Second-Harmonic Generation in the BBO Crystal	9
2.4.2	Crystal Rotation and Phase Matching	10
2.5	Camera Integration and Real-Time Data Acquisition	11
2.5.1	Camera Setup and Observations	11
2.5.2	Python Analysis	12
3	Software Analysis	13
3.1	Frame Extraction and Processing	13
3.2	Intensity Profile Analysis	13
3.3	Gaussian Fitting and Temporal Conversion	14
3.4	Data Visualization and Plotting	14
4	Results and Discussion	15
4.1	Pulse Duration Calculation	15
4.2	Gaussian Optimization through Motor Adjustment	16
4.3	Spectral Analysis	17
4.4	Conclusions and Future Work	18

List of Figures

1	Schematic of the SSA setup	4
2	Single-Shot Autocorrelator Implementation	5
3	Beam Splitter	6
4	Spatial Position of a SHG Fringe	7
5	Pulse Overlap in a BBO Crystal	9
6	Observed Fringe Pattern	10
7	Rotatable BBO Crystal	10
8	Recorded SHG Fringe Pattern	11
9	Intensity Profile of Recorded SHG Fringe	14
10	Bright Spatial Fringe	15
11	Intensity Profile for Bright Spatial Fringe	16
12	Gaussian after Motor Adjustment	16
13	Pulse after Motor Adjustment	17
14	Red Beam Spectrum	17

1 Introduction and History

1.1 Ultrashort Laser Pulses and Their Measurement Challenges

Ultrashort laser pulses, often on the femtosecond timescale (10^{-15} s), are the shortest events ever created and controlled by humans. Their utility lies in their ability to resolve processes on timescales that were previously inaccessible, as these pulses act as valuable tools for measuring ultrafast phenomena such as atomic motion, chemical reactions, and electron dynamics. This raises the question of whether these pulses can be made shorter. However, characterizing the temporal properties of a femtosecond pulse creates a paradox—in order to measure an event in time, a shorter event is required to measure it [2]. Tools such as photodiodes or oscilloscopes are too slow to capture these pulses; pulses cannot be measured using conventional methods as even the fastest detectors are too sluggish in comparison to the pulse duration. This prompts the need for a self-referencing method—the pulse measures itself.

1.2 Development of Autocorrelation Techniques

To address this challenge, an optical measurement technique called autocorrelation was developed to indirectly measure the pulse duration. The theoretical foundation for this was first introduced by Janszky et al., who proposed a framework for analyzing the temporal characteristics of short light pulses [4]. Building upon this, the first experimental demonstration of single-shot autocorrelation was performed by Salin et al., who used noncollinear second-harmonic generation (SHG) in a nonlinear crystal to transform temporal information into a spatial shape [1]. This is achieved by crossing two identical pulses within a nonlinear crystal, and introducing a time delay Δt_0 to one of the pulses. The temporal overlap of the pulses is mapped spatially, forming a SHG fringe pattern. This delay shifts the pattern along the x axis. The observed fringe can be converted into temporal information through a calibration technique.

Salin et al. describe the relationship between spatial displacement Δx_0 and temporal delay Δt_0 :

$$\Delta x_0 = \frac{c\Delta t_0}{2n \sin\left(\frac{\phi}{2}\right)},$$

where c is the speed of light, n is the refractive index of the nonlinear crystal, and ϕ is the angle between the crossing beams [1]. In practice, a calibration factor is experimentally determined to simplify this relationship, enabling direct conversion of spatial measurements (in pixels) to temporal delay (in femtoseconds). The spatial intensity distribution of the SHG fringe is typically Gaussian, making Gaussian fitting an effective technique for characterizing the pulse width [1]. The extracted intensity profiles from the SHG fringe are fitted to a Gaussian function to determine the full width at half maximum (FWHM), which is directly related to the temporal pulse width via the calibration factor.

1.3 The Single-Shot Autocorrelator

Figure 1 illustrates a basic setup of the single-shot autocorrelator (SSA), including the non-linear crystal and detection system. The SSA was chosen for this project due to its simple setup, and mathematical equivalence to traditional autocorrelation techniques [1]. This allows for reliable results without the need for complex autocorrelation systems.

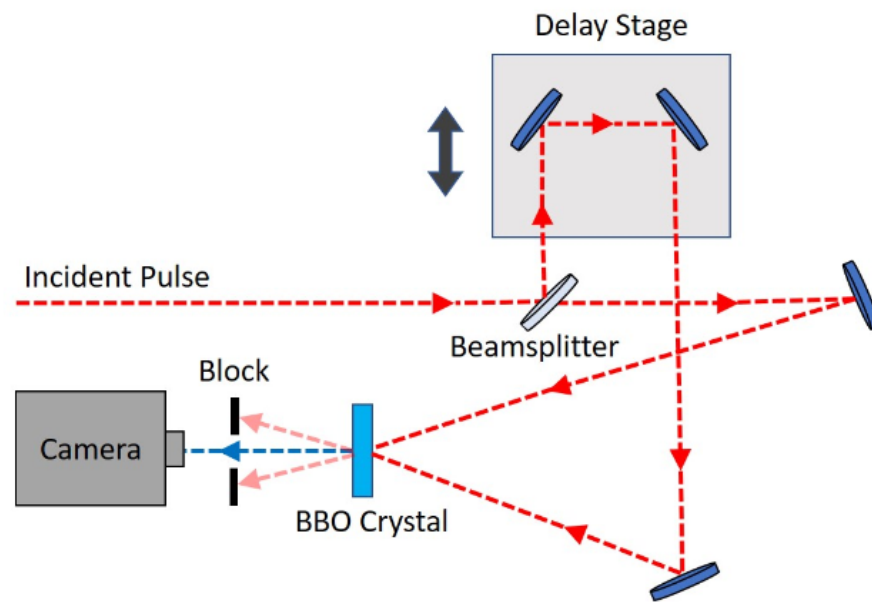


Figure 1: Schematic diagram of the single-shot autocorrelator [2].

The objectives of this project are to develop a reliable SSA setup to measure femtosecond pulses, and interface the autocorrelator with a camera system; integrating Python for real-time data acquisition.

2 Experimental Techniques

2.1 Single-Shot Autocorrelator Setup

The single-shot autocorrelator (SSA) was designed and built to measure the temporal properties of femtosecond laser pulses. An incident pulse is shot at the first mirror, which redirects it into the beam splitter. This creates two replicas of the initial pulse; one experiences a slight time delay through the delay stage, while the other is reflected by a mirror, unaffected by the delay. They then cross each other at the crystal, where a SHG pattern is formed. This pattern passes through blue filters to isolate the signal from the pulse replicas. Once isolated, the camera can record a spatial image of the pattern. [Figure 2](#) shows the path taken by an incident pulse after it is split into two replica beams.

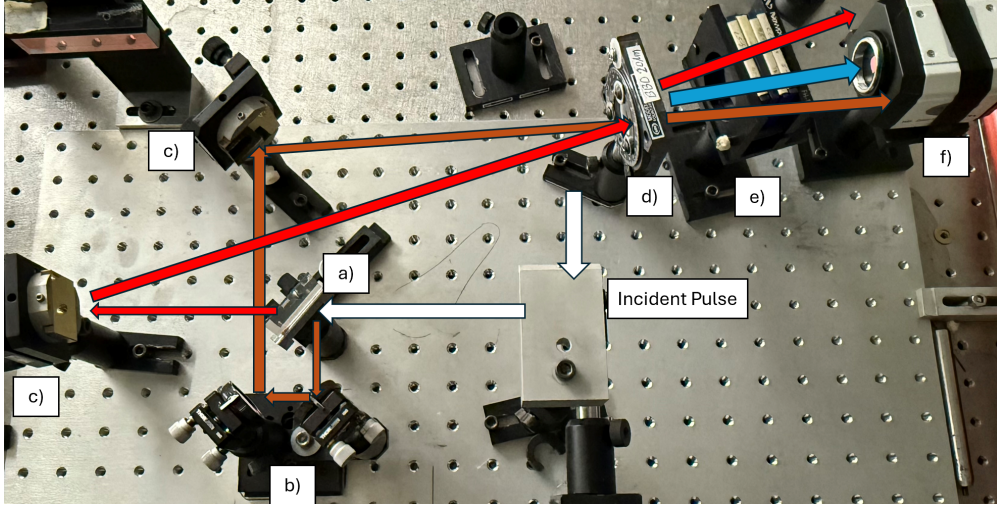


Figure 2: Implementation of the single-shot autocorrelator.

The components in [Figure 2](#) are as follows:

- (a) Beam splitter
- (b) Delay stage
- (c) Adjustable mirrors
- (d) Nonlinear BBO crystal
- (e) Several blue filters
- (f) CCD camera

Where the orange, red, and blue beams represent the delayed beam, undelayed beam, and autocorrelation signal, respectively.

2.2 Beam Alignment

Accurate beam alignment is important for achieving second-harmonic generation (SHG). A 50:50 beam splitter divides the incident laser pulse into two replicas of approximately equal intensity, pictured in Figure 3. These beams travel along separate paths to the nonlinear crystal (Figure 2) where they must to overlap both spatially and temporally to observe SHG.

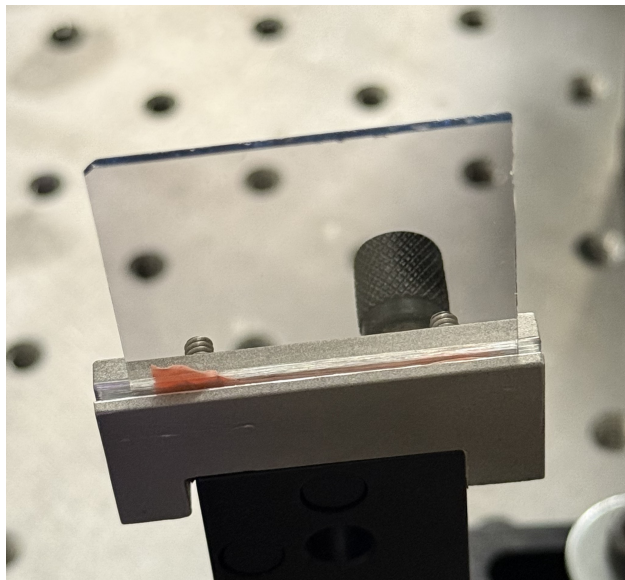


Figure 3: Polarized beam splitter.

To ensure temporal overlap, the path lengths of the split beams from the beam splitter to the nonlinear crystal were adjusted to be equal within the range of a few millimetres. This was done by physically moving the mirrors and measuring the path lengths using a string. Spatial overlap is achieved by adjusting the direction of the beam paths using the mirrors. The beams must intersect in the crystal at an optimal crossing angle ϕ (14 degrees in our setup), which influences both the phase-matching conditions and the system's temporal resolution [3]. This is influenced by the angle at which the mirrors are placed. Once the beams were aligned in space and time, a blue slit was observed, indicating SHG was correctly achieved.

2.3 Delay Stage

The delay stage is used to vary the optical path length of one of the two replicated beams after the beam splitter. A retroreflector is mounted on the moving platform of the delay stage to ensure that the beam is reflected back along its original path, regardless of small angular misalignments. When the delay stage is translated by a distance $\delta d_{\text{spatial}}$, the beam travels this extra distance twice — once toward the retroreflector and once back toward the beam splitter — resulting in a total optical path length change of $\Delta d_{\text{optical}} = 2\delta d_{\text{spatial}}$. This doubling must be accounted for when determining the temporal delay introduced.

By adjusting the micrometer screw on the delay stage, the temporal delay between the two beams can be precisely controlled. This delay causes a corresponding shift in the spatial position of the second-harmonic generation (SHG) fringe on the detector [1]. In the experimental setup, the fringe movement is monitored in real time using a Python-based intensity plot, providing a visual indicator of the fringe peak location. Figure 4 illustrates the intensity profile when the autocorrelation fringe is positioned at the extreme left edge of the camera’s field of view.

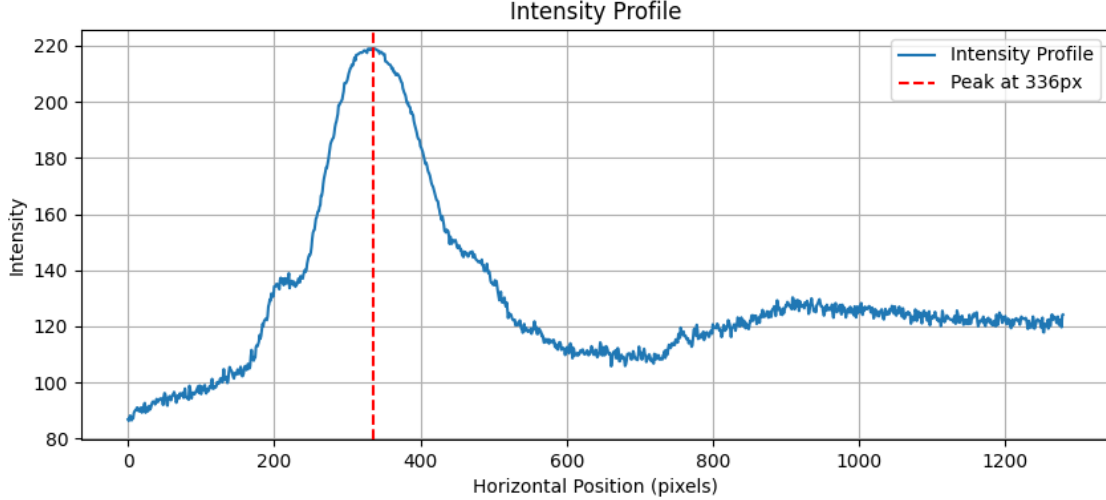


Figure 4: Spatial position of a recorded autocorrelation fringe. The peak intensity occurs at 336 horizontal pixels.

2.3.1 Derivation of the Temporal Calibration Factor

We calibrate time (fs) per pixel by relating the delay-stage motion to the optical path change and thus to a temporal delay. The key relation is

$$\Delta t = \frac{\Delta d_{\text{optical}}}{c},$$

where $\Delta d_{\text{optical}}$ is the *round-trip* optical path change (the retroreflector doubles the physical displacement), and c is the speed of light. As a useful check, $c \approx 30 \text{ cm/ns}$ (i.e., $1 \text{ }\mu\text{m}$ corresponds to $\approx 3.33 \text{ fs}$).

Measured pixel shift. The SHG fringe moved from 336 pixels to 976 pixels:

$$P = 976 - 336 = 640 \text{ pixels.}$$

Delay-stage motion. The micrometer was moved from 0.5 mm + 5 units to 0.5 mm + 24 units, where each unit is $1/50 \text{ mm} = 20 \text{ } \mu\text{m}$. Thus the *physical* displacement is

$$\delta d_{\text{spatial}} = (24 - 5) \times 20 \text{ } \mu\text{m} = 19 \times 20 \text{ } \mu\text{m} = 380 \text{ } \mu\text{m}.$$

Because the retroreflector changes the path length on both the forward and return legs, the *optical* path change is doubled:

$$\Delta d_{\text{optical}} = 2 \delta d_{\text{spatial}} = 760 \text{ } \mu\text{m}.$$

Temporal delay from path change. Using $1 \text{ } \mu\text{m} \leftrightarrow 3.33 \text{ fs}$,

$$\Delta t = \frac{\Delta d_{\text{optical}}}{c} = 760 \text{ } \mu\text{m} \times 3.33 \text{ fs}/\mu\text{m} = 2530.8 \text{ fs}.$$

Calibration factor (time per pixel).

$$C_f = \frac{\Delta t}{P} = \frac{2530.8 \text{ fs}}{640 \text{ px}} = \boxed{3.954375 \text{ fs/pixel}}.$$

2.3.2 Symmetry of the Autocorrelation and Stray Light

For a single-shot, noncollinear SHG autocorrelation, the ideal intensity $A(\tau)$ is an even function, $A(\tau) = A(-\tau)$, with a symmetric peak about zero delay. Any noticeable skew or asymmetry in the fringe or its extracted intensity profile typically indicates background contributions (e.g., stray light, scatter, or camera flare) or slight misalignment/phase-matching error. Practically, ensure:

- Good spatial filtering and adequate rejection of the fundamental (blue SHG isolation).
- Background subtraction (baseline removal) before fitting.
- Symmetry checks on the extracted profile; large asymmetry suggests contamination.

2.3.3 Energy/Frequency in SHG (for completeness)

In second-harmonic generation, two photons at angular frequency ω combine to produce one photon at 2ω (energy conservation: $\hbar\omega + \hbar\omega = \hbar(2\omega)$). This frequency doubling underpins the blue SHG signal used for the autocorrelation measurement.

2.4 Nonlinear Crystal and Phase Matching

In the single-shot autocorrelator setup, a beta barium borate (BBO) crystal with a thickness of 20 microns was used as the nonlinear medium for second-harmonic generation (SHG). The thickness of the crystal minimizes group velocity mismatch, which can broaden or distort the temporal profile of the SHG signal [5]. The orientation of the crystal is essential for observing SHG, as it ensures the signal remains in phase with the incoming pulses. Without phase matching, destructive interference occurs, and the generated signal diminishes rapidly.

2.4.1 Second-Harmonic Generation in the BBO Crystal

Second-harmonic generation (SHG) occurs when two photons of the same frequency ω meet within the nonlinear crystal to produce a photon with double the frequency 2ω , due to conservation of momentum. This generates an interference pattern that varies with the pulse duration. Shorter pulses create narrower fringes, whereas longer pulses produce broader fringes [5]. The SHG signal is generated only where the two pulses coincide both spatially and temporally, with the strongest signal occurring at zero delay, as shown in Figure 5. The observed fringe pattern is shown in Figure 6.

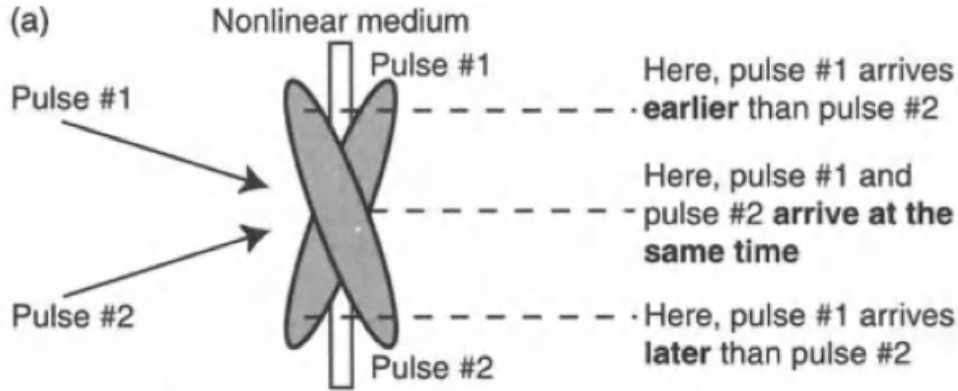


Figure 5: Visualization of pulse overlap in the BBO crystal. The SHG signal is most intense where the centers of both pulses coincide in time. Adapted from [2].

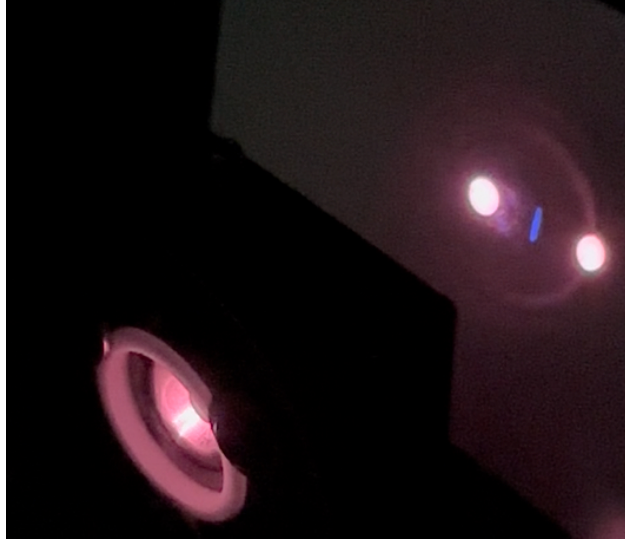


Figure 6: Observed SHG fringe pattern produced by the 20-micron BBO crystal. The blue slit indicates where SHG occurs. The red circles are the two pulses generated by the beam splitter.

2.4.2 Crystal Rotation and Phase Matching

To achieve phase matching, the BBO crystal was mounted on a rotatable holder, allowing for adjustment of its orientation, as shown in Figure 7. The crystal was rotated while monitoring the resulting fringe pattern on the camera. The optimal orientation was determined by observing the thinnest fringes, which correspond to the shortest pulse durations [5].

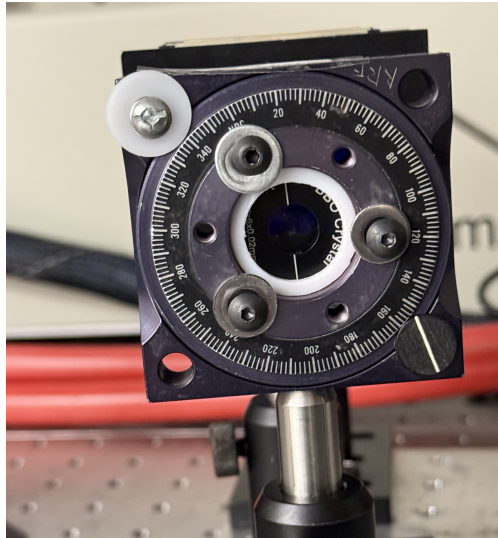


Figure 7: Rotatable BBO crystal used to optimize phase matching.

2.5 Camera Integration and Real-Time Data Acquisition

2.5.1 Camera Setup and Observations

The single-shot autocorrelator setup integrates a CCD camera positioned behind the blue filters to capture the SHG fringe pattern, as illustrated in [Figure 2](#). The blue light filters isolate the SHG signal from the input pulses. This is done to improve the signal-to-noise ratio, as stray light from the input pulses results in background noise that can obscure the signal and reduce the accuracy of the intensity profile analysis.

The camera is connected to a laptop running a Python script, which displays the captured autocorrelation signal in real time. The real-time feedback allows for immediate visualization of the SHG fringe pattern, as shown in [Figure 8](#). This visualization is particularly helpful when aligning the optical components, as any changes made to the setup will be reflected in the signal's shape.

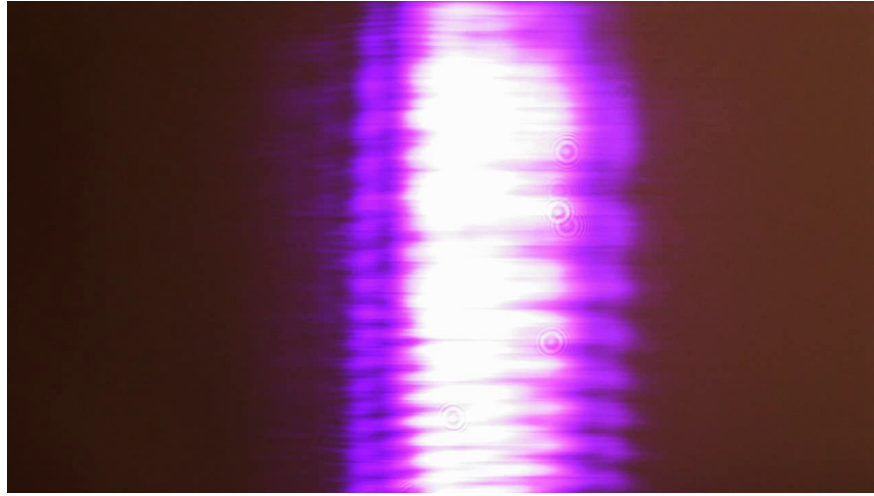


Figure 8: SHG fringe pattern captured by the camera.

2.5.2 Python Analysis

The data acquisition and analysis in Python follow these steps:

1. **Frame Capture:**

The script records a video of the pulse duration using the camera. The video is then split into frames and stored in a folder named "Frames".

2. **Intensity Profile Extraction:**

An intensity profile is extracted along the central row of each frame. This reduces noise and focuses on the main features of the trace.

3. **Gaussian Fitting:**

A Gaussian function is fitted to the extracted intensity profile to determine the full width at half maximum (FWHM) in pixels. This characterizes the pulse shape based on the spatial distribution of the fringe.

4. **FWHM to Time Conversion:**

The FWHM value in pixels is converted to femtoseconds using the temporal calibration factor derived from the delay stage. This step allows the pulse duration to be expressed in units of time.

5. **Visualization and Storage:**

The intensity profiles, Gaussian fits, and calculated pulse durations are visualized in real time. Plots of the processed data are saved in the same directory as the video and frames, in a folder named "Plots".

3 Software Analysis

3.1 Frame Extraction and Processing

The script presents a real-time plot of the observed intensity by the connected camera. Once a desired fringe is observed, a button to prompt a recording can be pressed to record a video of the camera feed. This is recorded at 1280×720 pixels, at 15 frames per second. After recording, the video is split into frames and processed for intensity analysis.

3.2 Intensity Profile Analysis

The intensity profile is obtained by averaging several rows around the center of the image. This averaging reduces noise and focuses on the main features of the autocorrelation trace:

$$\text{Intensity Profile} = \frac{1}{N} \sum_{i=\text{Center}-R}^{\text{Center}+R} \text{Row}[i]$$

Figure 4 illustrates the intensity profile of a SHG fringe. The extracted profile represents the spatial intensity distribution of the SHG signal, with pixel intensity values corresponding to the brightness levels in the image. These intensity values are based on the greyscale pixel values of the image, which range from 0 (black) to 255 (white).

The input SHG fringe images are converted to greyscale to ensure that only the brightness information is used, eliminating possible complexities from color data. The rows of the greyscale image are represented as 2D arrays of pixel intensity values, with each pixel corresponding to a spatial position in the fringe. By averaging several rows around the central region of interest, the intensity profile captures the overall shape of the fringe while suppressing high-frequency noise.

A Gaussian filter is applied to the intensity profile to further reduce noise and smooth out sharp variations caused by random fluctuations in the camera's sensor. This process involves using a Gaussian-shaped function to smooth the intensity data:

$$\text{Smoothed Profile} = \text{Gaussian Filter}(\text{Intensity Profile}, \sigma)$$

In this formula, σ is the standard deviation of the Gaussian function, which determines how much smoothing is applied. A larger σ value provides better smoothing of the profile but may also blur important details. This step allows for easier fitting of a Gaussian curve to the intensity profile, minimizing environmental noise.

3.3 Gaussian Fitting and Temporal Conversion

The FWHM of a Gaussian is related to its standard deviation (σ) by (see appendix):

$$\text{FWHM} = 2\sqrt{2\ln(2)} \cdot \sigma \approx 2.3548 \cdot \sigma$$

This is calculated from the intensity profile extracted from the SHG fringe. A Gaussian curve is then fitted to the intensity profile using a curve-fitting function from the SciPy library. This provides the values for amplitude, center position, and standard deviation, which allows for the FWHM to be calculated in pixels.

Once the spatial FWHM is obtained, it is converted to temporal FWHM using the experimentally determined calibration factor (C_f), which links pixels to femtoseconds:

$$\Delta t_{\text{FWHM}} = C_f \left(\frac{\text{fs}}{\text{pixel}} \right) \cdot \text{FWHM (pixels)}.$$

By using this relation, the pulse duration is obtained in femtoseconds.

3.4 Data Visualization and Plotting

Once the FWHM is calculated for each frame, the results are plotted and saved for offline analysis. [Figure 9](#) shows the processed intensity profile of [Figure 8](#) with the Gaussian fit and spatial FWHM.

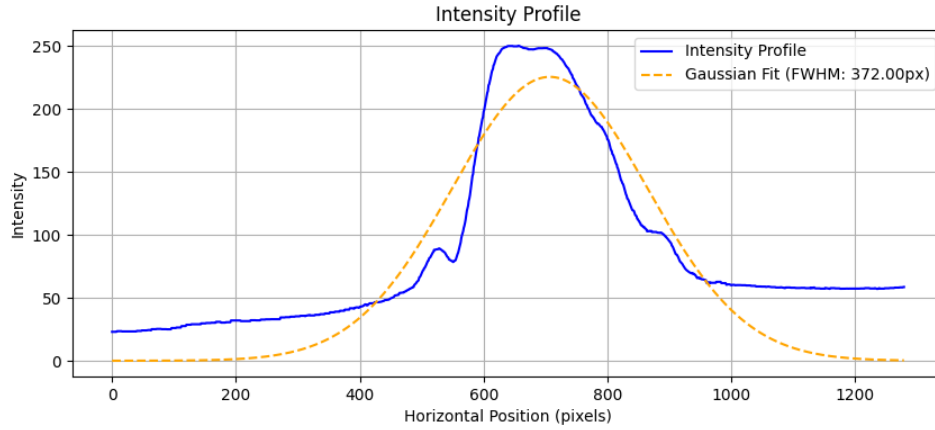


Figure 9: Intensity profile of [Figure 8](#) with a Gaussian fit. The FWHM is calculated in pixels.

4 Results and Discussion

4.1 Pulse Duration Calculation

The autocorrelation fringe with the best fitting Gaussian curve (pictured in [Figure 10](#)) for the observed intensity profile yielded a FWHM of 307.15 pixels. Using the calibration factor determined earlier, $C_f = 3.954375$ fs/pixels, the temporal duration of the pulse was calculated using the relationship:

$$\Delta t_{\text{FWHM}} = \text{FWHM}_{\text{pixels}} \times C_f$$

where Δt_{FWHM} is the pulse duration in femtoseconds. Substituting the measured FWHM:

$$\Delta t_{\text{FWHM}} = 307.15 \text{ pixels} \times 3.954375 \frac{\text{fs}}{\text{pixel}} \approx 1214.586281 \text{ fs}.$$

Thus, the duration of the femtosecond laser pulse was determined to be approximately 1.21 ps. This result shows that the single-shot autocorrelator is capable of characterizing ultrashort laser pulses with high temporal precision, however, this is far from the expected value of 30 fs.

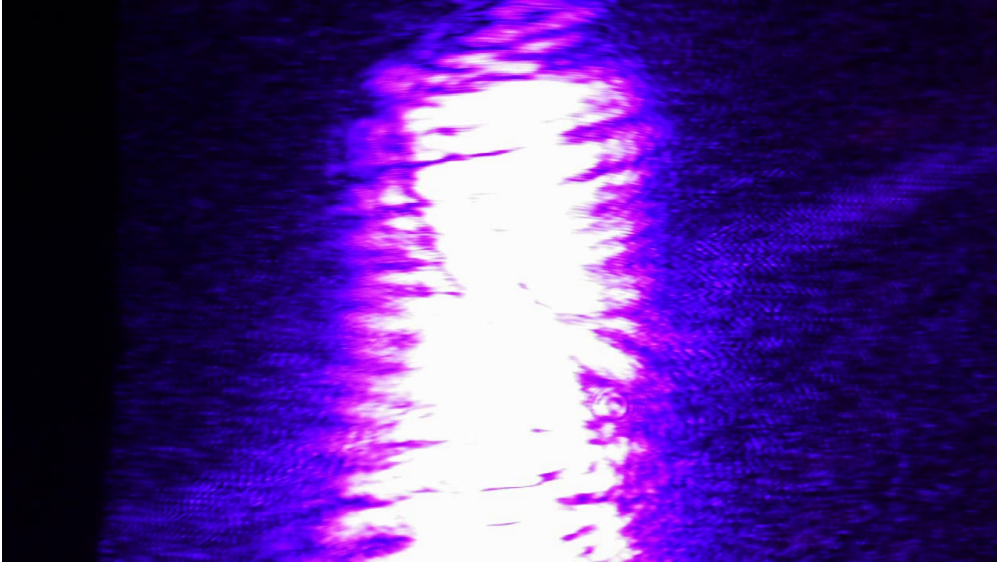


Figure 10: Spatial SHG fringe pattern recorded by the camera.

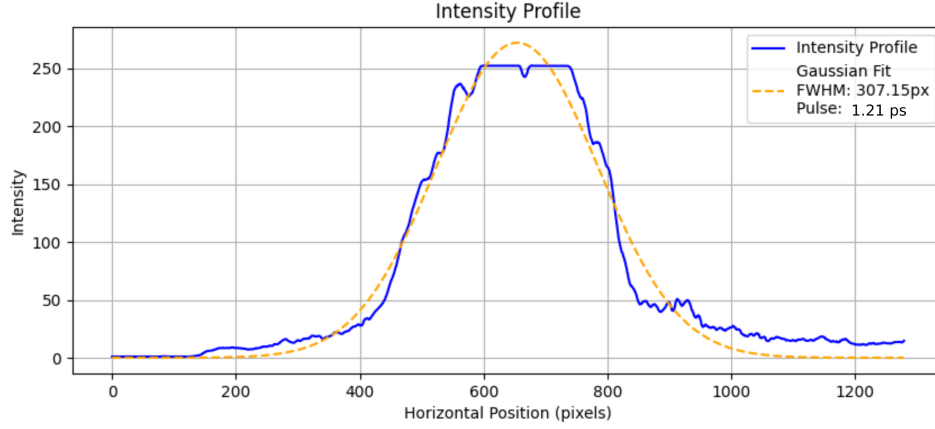


Figure 11: Intensity profile extracted from the spatial SHG fringe pattern with a fitted Gaussian curve. The spatial FWHM and calculated pulse duration are shown.

4.2 Gaussian Optimization through Motor Adjustment

Initial measurements of the pulse duration yielded relatively large values on the order of picoseconds, indicating suboptimal pulse compression. By adjusting the motor position controlling the angle of the compressor grating from 5585 to 4886, the Gaussian profile of the autocorrelation signal improved significantly. This adjustment not only reduced the measured pulse duration but also resulted in a visibly narrower beam profile on the camera, transitioning from a broad distribution to a more confined spatial profile.

Figure 12 shows the improved Gaussian profile, with a FWHM corresponding to a pulse duration of 90.75 fs — a substantial improvement compared to the initial measurement of 1.21 ps.

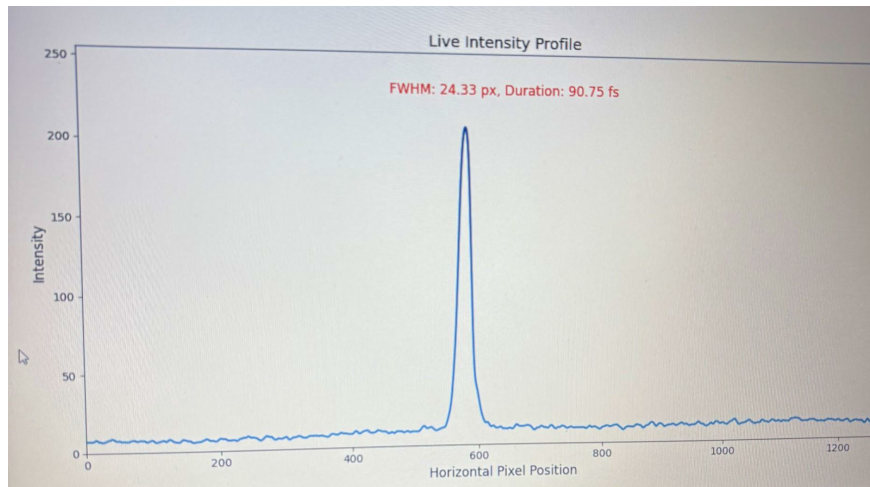


Figure 12: Gaussian fit of the pulse after decreasing the compressor grating angle. The spatial FWHM and calculated pulse duration are shown.

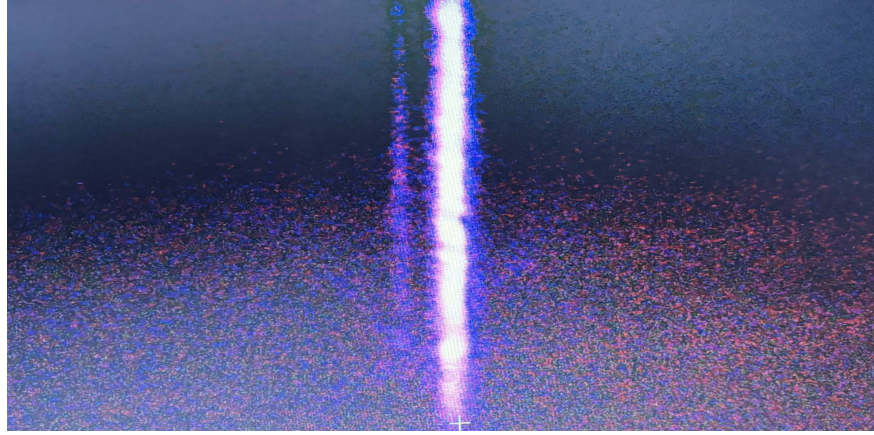


Figure 13: Image of the beam profile after decreasing the compressor grating angle.

4.3 Spectral Analysis

Following the grating adjustment, the optical spectrum was recorded and compared to the initial red beam spectrum. Theoretically, a shorter pulse duration should correspond to a broader spectral bandwidth, in accordance with the Fourier transform limit. However, in this case, the spectrometer distance from the beam was not kept constant between measurements, introducing a source of error in the comparison. This means the observed spectral difference is not necessarily representative of the true change in bandwidth.

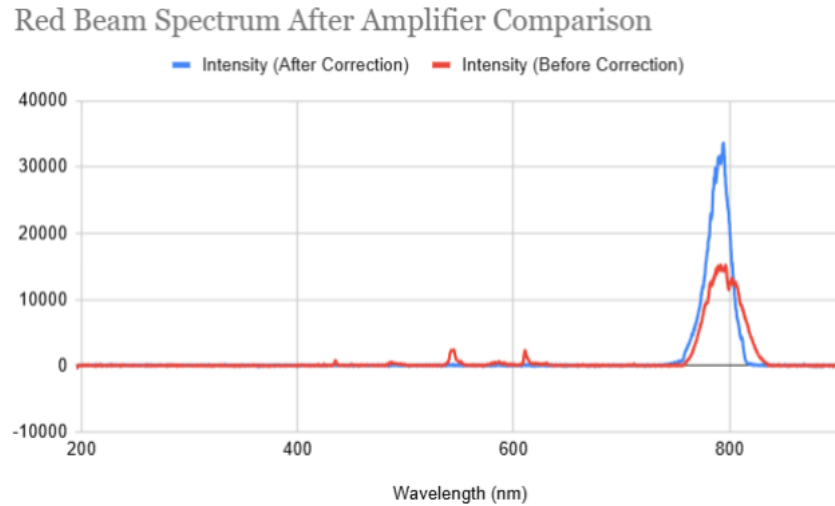


Figure 14: Comparison of measured spectra before and after compressor grating adjustment.

To ensure accurate spectral comparison in future measurements, both spectra should be acquired with the spectrometer positioned at the same fixed distance from the beam. This will ensure that observed differences are attributable to pulse compression rather than variations in collection geometry.

4.4 Conclusions and Future Work

The single-shot autocorrelator successfully demonstrated femtosecond pulse characterization using noncollinear second-harmonic generation (SHG) and Gaussian fitting. Initial measurements yielded pulse durations in the picosecond range, indicating suboptimal compression. Following the adjustment of the compressor grating angle from 5585 to 4886 motor units, the measured autocorrelation trace improved significantly, achieving a pulse duration of 90.75 fs as shown in [Figure 12](#). This improvement was accompanied by a visible narrowing of the beam profile in [Figure 13](#), consistent with the expected behavior for shorter pulses. The clear enhancement in Gaussian fitting precision motivated a follow-up spectral analysis to verify whether the observed pulse shortening corresponded to a broadened optical spectrum.

The spectral results, shown in [Figure 14](#), were inconclusive due to inconsistent spectrometer positioning between the two measurements, preventing a reliable comparison of spectral widths. This inconsistency underscores the need for maintaining a fixed spectrometer position when comparing different compression settings. In addition, background noise and stray light—visible in some spatial images as diagonal streaks—can distort the Gaussian fit and introduce asymmetry into the autocorrelation trace [5], affecting measurement accuracy.

Future improvements should therefore focus on maintaining consistent spectrometer alignment to enable accurate bandwidth measurements, enhancing stray-light rejection through improved optical filtering and isolation of the detection path, exploring alternative nonlinear crystals or optimizing crystal orientation to produce finer fringe patterns, and upgrading the imaging system to a higher-resolution, lower-noise camera. With these refinements, the autocorrelator will be capable of delivering more precise pulse duration measurements and more reliable spectral verification of pulse compression.

Acknowledgments

I would like to acknowledge my supervisor, Dr. Joseph Sanderson, for providing me with the opportunity to participate in this research. His constant support throughout the term helped me become a much better researcher. I would like to also thank Dr. Reza Karimi for his support in the lab throughout the term, especially regarding laser configurations and the autocorrelator setup. His insight provided me with a lot of background knowledge on the experimental side of the project, and I feel I have learned a lot as a result. I would like to thank Zhiyu Zhao for his help during the setup of the autocorrelator, fringe observations, and our long conversations about the theory of the papers we read. Finally, I would like to acknowledge Aryan Sood and Rohan Bhatt for their contributions to achieving an improved pulse measurement by adjusting the compressor and recording the data.

References

- [1] F. Salin, P. Georges, G. Roger, and A. Brun, “Single-shot measurement of a 52-fs pulse,” *Applied Optics*, vol. 26, no. 21, pp. 4528–4531, 1987.
- [2] M. Singh, “Few-cycle laser pulse generation and characterization for Coulomb explosion imaging of molecules,” Master’s thesis, University of Waterloo, 2019.
- [3] C. Rullière, *Femtosecond Laser Pulses: Principles and Experiments*, 2nd ed., Springer, 2005.
- [4] J. Janszky, G. Corradi, and R. N. Gyuzalian, “On a possibility of analysing the temporal characteristics of short light pulses,” *Optics Communications*, vol. 23, no. 3, pp. 293–298, 1977.
- [5] R. Trebino, *Frequency-Resolved Optical Gating: The Measurement of Ultrashort Laser Pulses*, Springer, 2000.

Appendix

A: Gaussian Factor Derivation

The Gaussian function is defined as:

$$G(x) = A \exp \left(-\frac{(x - x_0)^2}{2\sigma^2} \right),$$

where A is the amplitude of the Gaussian, x_0 is the center of the Gaussian, and σ is the standard deviation. The FWHM is the width of the Gaussian curve at half of its maximum value, i.e., $G(x) = A/2$.

$$\begin{aligned} \frac{A}{2} &= A \exp \left(-\frac{(x - x_0)^2}{2\sigma^2} \right) \\ \frac{1}{2} &= \exp \left(-\frac{(x - x_0)^2}{2\sigma^2} \right) \\ \ln \left(\frac{1}{2} \right) &= -\frac{(x - x_0)^2}{2\sigma^2} \\ -\ln(2) &= -\frac{(x - x_0)^2}{2\sigma^2} \\ (x - x_0)^2 &= 2\sigma^2 \ln(2) \\ x - x_0 &= \pm \sqrt{2\sigma^2 \ln(2)} \end{aligned}$$

The FWHM is the total width between the two points at half maximum, expressed as:

$$\text{FWHM} = 2|x - x_0| = 2\sqrt{2\sigma^2 \ln(2)} = 2\sigma\sqrt{2\ln(2)}.$$

The numerical value of $\sqrt{2\ln(2)}$ is approximately 1.1774. Substituting this value:

$$\text{FWHM} = 2 \cdot 1.1774 \cdot \sigma \approx 2.3548 \cdot \sigma.$$

Thus, the FWHM of a Gaussian function is related to its standard deviation σ by:

$$\text{FWHM} = 2.3548 \cdot \sigma.$$

B: Python Code for SSA Analysis

```
import cv2
import numpy as np
from tkinter import Tk, Button, Frame, filedialog
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg
import matplotlib.pyplot as plt
from scipy.ndimage import gaussian_filter
from scipy.optimize import curve_fit
import os

# Calibration factor (fs/pixel)
CALIBRATION_FACTOR = 3.954375

# Camera setup
width = 1280
height = 720
frame_rate = 15
camera_index = 2 # Change to 0 for built-in webcam, 2 for external camera
vid_capture = cv2.VideoCapture(camera_index, cv2.CAP_DSHOW)
vid_capture.set(3, width)
vid_capture.set(4, height)

# Recording setup
output = None # Video writer object
is_recording = False # Flag for recording state
output_dir = None
update_after_id = None # Store the ID for root.after calls

# Tkinter setup
root = Tk()
root.title("Camera Feed with Toolbar")
root.geometry("1360x800") # Adjust window size

# Matplotlib figure for live intensity profile
fig, ax = plt.subplots(figsize=(6, 4))
line, = ax.plot([], [], lw=2) # Line object for the intensity profile
fwhm_text = ax.text(0.5, 0.9, '', transform=ax.transAxes, fontsize=10,
    ↪ ha='center', color='red')
ax.set_xlim(0, width) # X-axis range based on frame width
ax.set_ylim(0, 255) # Y-axis range for intensity values (grayscale)
ax.set_xlabel("Horizontal Pixel Position")
ax.set_ylabel("Intensity")
ax.set_title("Live Intensity Profile")
```

```

# Embed Matplotlib plot in Tkinter
canvas = FigureCanvasTkAgg(fig, master=root)
canvas_widget = canvas.get_tk_widget()
canvas_widget.pack(side="top", fill="both", expand=True)

# Intensity profile smoothing
def extract_intensity_profile(frame):
    """Extracts and smooths the intensity profile from a grayscale image."""
    center_row = frame.shape[0] // 2
    row_range = 20 # Number of rows to average
    intensity_profile = np.mean(frame[center_row - row_range:center_row +
    ↪ row_range, :], axis=0)
    smoothed_profile = gaussian_filter(intensity_profile, sigma=2)
    return smoothed_profile

# Frame analysis and Gaussian fitting
def analyze_frame(image_path, plot_dir):
    """Performs Gaussian fitting on intensity profiles and saves results."""
    image = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
    if image is None:
        print(f"Image {image_path} could not be read.")
        return

    intensity_profile = extract_intensity_profile(image)

    # Define Gaussian function
    def gaussian(x, amp, center, sigma):
        return amp * np.exp(-(x - center)**2 / (2 * sigma**2))

    x_data = np.arange(len(intensity_profile))
    initial_guess = [intensity_profile.max(), len(intensity_profile) / 2, 10]
    try:
        popt, _ = curve_fit(gaussian, x_data, intensity_profile,
        ↪ p0=initial_guess)
        sigma_fit = popt[2]
        fwhm_pixels = 2 * np.sqrt(2 * np.log(2)) * sigma_fit
        pulse_duration = fwhm_pixels * CALIBRATION_FACTOR
        print(f"FWHM (pixels): {fwhm_pixels:.2f}, Pulse Duration (fs):
        ↪ {pulse_duration:.2f}")

    # Save Gaussian fit plot
    plt.figure()
    plt.plot(x_data, intensity_profile, label="Intensity Profile")
    plt.plot(x_data, gaussian(x_data, *popt), label=f"Gaussian Fit (FWHM:
    ↪ {fwhm_pixels:.2f}px, Duration: {pulse_duration:.2f} fs)",
    ↪ linestyle="--", color="red")
    plt.xlabel("Pixel Position")

```

```

plt.ylabel("Intensity")
plt.title("Gaussian Fit to Intensity Profile")
plt.legend()
plot_path = os.path.join(plot_dir,
    ↪ f"{os.path.basename(image_path).split('.')[0]}_fit.png")
plt.savefig(plot_path)
plt.close()
except Exception as e:
    print(f"Error fitting Gaussian to {image_path}: {e}")

# Extract frames from video
def extract_frames(video_path, frame_dir):
    """Extracts frames from the recorded video."""
    cap = cv2.VideoCapture(video_path)
    count = 0
    while cap.isOpened():
        ret, frame = cap.read()
        if not ret:
            break
        frame_path = os.path.join(frame_dir, f"frame_{count:05d}.jpg")
        cv2.imwrite(frame_path, frame)
        count += 1
    cap.release()
    print(f"{count} frames extracted to {frame_dir}.")

# Update function for live graph and camera feed
def update():
    global is_recording, output, output_dir, update_after_id
    ret, live_frame = vid_capture.read()
    if not ret:
        return

    # Convert to grayscale for intensity profile
    gray_frame = cv2.cvtColor(live_frame, cv2.COLOR_BGR2GRAY)
    intensity_profile = extract_intensity_profile(gray_frame)

    # Perform real-time Gaussian fitting for live FWHM calculation
    def gaussian(x, amp, center, sigma):
        return amp * np.exp(-(x - center)**2 / (2 * sigma**2))

    x_data = np.arange(len(intensity_profile))
    initial_guess = [intensity_profile.max(), len(intensity_profile) / 2, 10]
    try:
        popt, _ = curve_fit(gaussian, x_data, intensity_profile,
            ↪ p0=initial_guess)
        sigma_fit = popt[2]
        fwhm_pixels = 2 * np.sqrt(2 * np.log(2)) * sigma_fit

```

```

        fwhm_text.set_text(f"FWHM: {fwhm_pixels:.2f} px")
except:
    fwhm_text.set_text("FWHM: N/A")

# Update the line on the live graph
line.set_data(np.arange(len(intensity_profile)), intensity_profile)
canvas.draw()

# Display the camera feed
cv2.imshow("Camera Feed", live_frame)

# Record video if enabled
if is_recording and output:
    output.write(live_frame)

# Schedule the next update
update_after_id = root.after(50, update)
# Start recording
def start_recording():
    global is_recording, output, output_dir
    if not is_recording:
        output_dir = filedialog.askdirectory(title="Select Save Directory")
        if not output_dir:
            print("No directory selected.")
            return
        vid_name = "Recorded_Video"
        video_file = os.path.join(output_dir, vid_name + ".mp4")
        os.makedirs(output_dir, exist_ok=True)
        global frame_dir, plots_dir
        frame_dir = os.path.join(output_dir, "Frames")
        plots_dir = os.path.join(output_dir, "Plots")
        os.makedirs(frame_dir, exist_ok=True)
        os.makedirs(plots_dir, exist_ok=True)
        output = cv2.VideoWriter(video_file, cv2.VideoWriter_fourcc(*'mp4v'),
            ↪ frame_rate, (width, height))
        is_recording = True
        print("Recording started.")

# Stop recording
def stop_recording():
    global is_recording, output
    if is_recording:
        is_recording = False
        if output:
            output.release()
            output = None
        print("Recording stopped. Analyzing frames...")

```

```

        extract_frames(os.path.join(output_dir, "Recorded_Video.mp4"),
            ↪ frame_dir)
    for frame_file in os.listdir(frame_dir):
        frame_path = os.path.join(frame_dir, frame_file)
        analyze_frame(frame_path, plots_dir)

# Exit program
def exit_program():
    global vid_capture, output, update_after_id
    if update_after_id:
        root.after_cancel(update_after_id) # Cancel the scheduled update
    if output:
        output.release()
    vid_capture.release()
    cv2.destroyAllWindows()
    root.destroy()

# Toolbar buttons
toolbar = Frame(root)
toolbar.pack(side="bottom", fill="x")

start_button = Button(toolbar, text="Start Recording", command=start_recording)
start_button.pack(side="left", padx=10, pady=5)

stop_button = Button(toolbar, text="Stop Recording", command=stop_recording)
stop_button.pack(side="left", padx=10, pady=5)

exit_button = Button(toolbar, text="Exit", command=exit_program)
exit_button.pack(side="left", padx=10, pady=5)

# Start updating
update_after_id = root.after(50, update)

# Start the Tkinter main loop
root.mainloop()

```
